VIETNAM NATIONAL UNIVERSITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Deep Learning And Applied Computer Vision

EXERCISES

Instructor(s): Le Thanh Sach

Group Members:

Tran Ngoc Dat - 2570388

Nguyen Dinh Quang Khai - 2570231

HO CHI MINH, October, 2026

Table Contents

Table Contents 3
1. Dataset 5
1.1 CIFAR-10 Dataset Overview 5
2. Exercises 6
2.1 E1 6
2.1.1 SoftMax Classifier 6
2.1.3.1 Introduction 6
2.1.3.3 Discussion And Experimental Analysis 7
2.1.3.3.1 Softmax Configuration 7
2.1.3.3.2 Softmax Definition Layers 7
2.1.3.3.1.1 Softmax Forward Implementation 7
2.1.3.3.2 Analysis 7
2.1.2 Multilayer Perceptron Classifier 9
2.1.2.1 Introduction 9
2.1.2.2 Discussion And Experimental Analysis 10
2.1.2.2.1 Multilayer Perceptron Classifier Configuration 10
2.1.2.2.1.1 MLP Definition Layers 10
2.1.2.2.1.2 MLP Forward Implementation 10
2.1.2.2.2 Analysis 11
2.1.3 Convolutional Neural Network Classifier 14
2.1.3.1 Introduction 14
2.1.3.2 Discussion And Experimental Analysis 15
2.1.3.2.1 Convolutional Neural Network Configuration 15
2.1.3.2.1.1 CNN Definition Layers 15
2.1.3.2.1.2 CNN Forward Implementation 16
2.1.3.2.2 Analysis 17
2.1.4 The Comparison And Evaluation 20
2.1.4.1 Learning Curve 20
2.1.4.1 Classification Metric 21
2.2 E2 23
2.2.1 Introduction 23
2.2.2 Multi-Head Self-Attention (MSA) 23
2.2.3 ViT with Custom MSA Definition Layers (Patch Token 4x4) 24
2.2.4 ViT with Custom MSA Forward Implementation Flow 24
2.2.5 Hyperparameters Configuration and Architectural Justifications 25
2.2.6 MSA Implementation 26
2.2.7 Result Analyzing 28
2.3 E3 30
2.3.1 Long Short-Term Memory (LSTM) 30
2.3.1.1 Introduction 30

2.3.1.2 Discussion And Experimental Analysis.....32
2.3.1.2.1 Long Short-Term Memory (LSTM) Configuration.....32
2.3.1.2.1.1 LSTM Definition Layers.....32
2.3.1.2.1.2 LSTM Forward Implementation.....32
2.3.1.2.2 Analysis.....33
2.3.2 Gated Recurrent Unit (GRU).....35
2.3.2.1 Introduction.....35
2.3.2.2 Discussion And Experimental Analysis.....37
2.3.2.2.1 Gated Recurrent Unit Configuration.....37
2.3.2.2.1.1 GRU Definition Layers.....37
2.3.2.2.1.2 GRU Forward Implementation.....38
2.3.2.2.2 Analysis.....38
2.3.3 The Comparison And Evaluation Between LSTM And GRU.....41
2.3.4 The Comparison And Evaluation Between LSTM And MLP.....42
2.3.5 The Comparison And Evaluation Between LSTM And CNN.....43
2.3.5 The Comparison And Evaluation Between GRU And MLP.....43
2.3.6 The Comparison And Evaluation Between GRU And CNN.....44
3. References.....44

1. Dataset

1.1 CIFAR-10 Dataset Overview

The **CIFAR-10** dataset is a standard computer vision dataset. It consists of 60,000 RGB (3 Channels) images, 32x32 pixels, containing one of 10 object classes, with 6,000 images per class.
The **CIFAR-10 dataset** collected by Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton.
There are 50,000 training images, and 10,000 test images

airplane										
automobile										
bird										
cat										
deer										
dog										
frog										
horse										
ship										
truck										

The images 10 classes of CIFAR10 dataset

Split Configuration:

Total (images)	Train Set (images)	Validation Set (images)	Test Set (images)
60000	45000	5000	10000

2. Exercises

2.1 E1

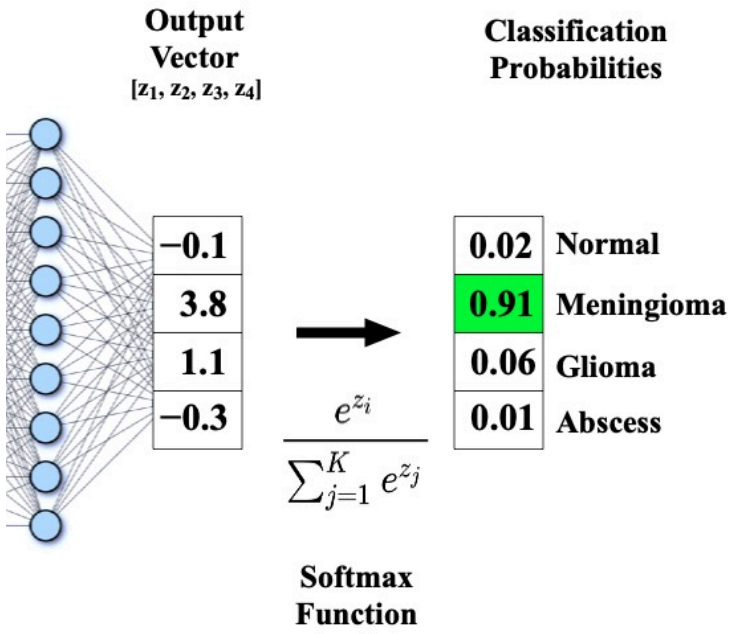
2.1.1 SoftMax Classifier

2.1.3.1 Introduction

A **softmax classifier** (also known as multinomial logistic regression or maximum entropy classifier) is a machine learning model used for multi-class classification

How does the Softmax Classifier Work?
In a Softmax Classifier, the neural network outputs a set of raw scores for each class. These raw scores, also called logits, are then passed through the Softmax function, converting them into probabilities. The class with the highest probability is chosen as the model's prediction.

Softmax Function: $p_i = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_j}}$



The visualization of Softmax Classifier

2.1.3.3 Discussion And Experimental Analysis

2.1.3.3.1 Softmax Configuration

2.1.3.3.2 Softmax Definition Layers

Layer	Input	Output
Flatten	tensor([3,32,32))	vector([x1, x2,...,x3072]); 3072 pixels
Linear: Linear(3 * 32 * 32, num_classes)	vector([x1, x2,...,x3072]); 3072 pixels	a vector of 10 logits, with each logit corresponding to a class

2.1.3.3.1.1 Softmax Forward Implementation

Order	Layer
1	**Flatten**
2	**Fully Connected**

2.1.3.3.2 Analysis

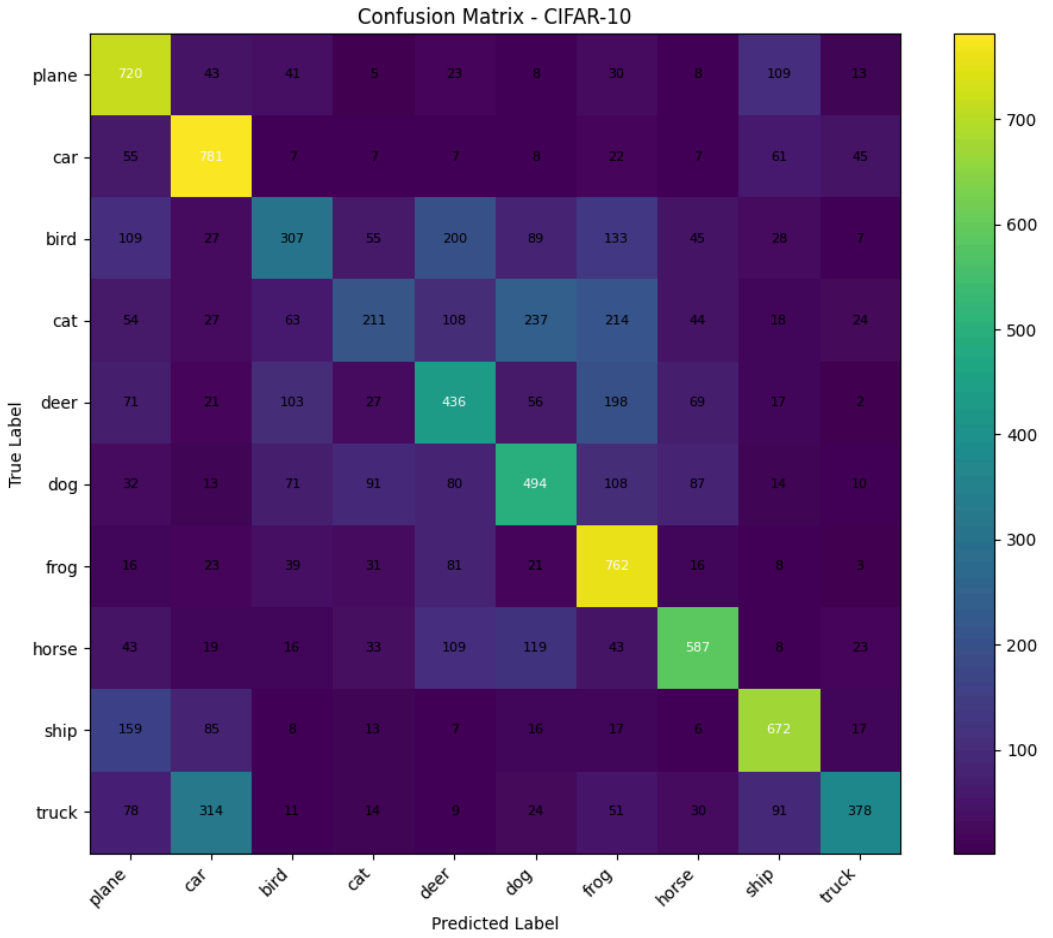
Learning Rate	Batch Size	Train Loss	Test Loss	Train Accuracy	Test Accuracy
0.001	128	1.2446	1.2826	55.56%	53.48%

The table accuracy of **10 classes** based on the **Test Dataset**

airplane	**72.0%**
automobile	**78.3%**
bird	**30.7%**
cat	**21.1%**
deer	**43.6%**
dog	**49.4%**
frog	**76.0%**
horse	**58.8%**

ship	**67.2%**
truck	**37.9%**

Confusion Matrix



The Confusion Matrix of Softmax Model

Classification Metric:

Accuracy (%)	Precision	Recall	F1-Score
45.27	0.5		0.

classes	precision	recall	f1-score	samples
airplane	0.54	0.72	0.62	1000
automobile	0.58	0.78	0.66	1000
bird	0.46	0.31	0.37	1000
cat	0.43	0.21	0.28	1000

deer	0.41	0.44	0.42	1000
dog	0.46	0.49	0.48	1000
frog	0.48	0.76	0.59	1000
horse	0.65	0.59	0.62	1000
ship	0.65	0.67	0.66	1000
truck	0.72	0.38	0.50	1000

	precision	**recall**	**f1-score**	**samples**
accuracy			0.53	10000
macro avg	0.54	0.53	0.52	10000
weighted avg	0.54	0.53	0.52	10000

Display Misclassified Images based on Classes

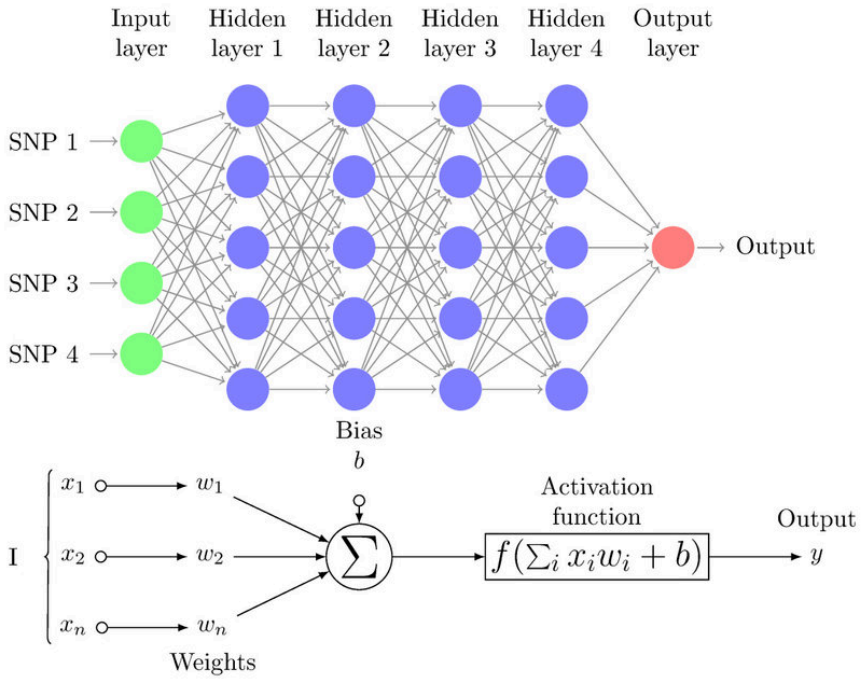
2.1.2 Multilayer Perceptron Classifier

2.1.2.1 Introduction

A Multi-Layer Perceptron is a type of neural network that consists of multiple layers of nodes, or "neurons." These layers include -

- Input Layer: This is where the data enters the network. Each neuron in this layer represents one feature of the input data.
- Hidden Layers: These are intermediate layers between the input and output layers. An MLP can have one or more hidden layers, which allow the network to learn complex patterns.
- Output Layer: This layer produces the final result, such as a class label or a numeric value, depending on the task at hand.

Each neuron in a layer is connected to every neuron in the next layer, and these connections have weights that are adjusted during training to improve the network’s performance.



The MLP Architecture

2.1.2.2 Discussion And Experimental Analysis

2.1.2.2.1 Multilayer Perceptron Classifier Configuration

2.1.2.2.1.1 MLP Definition Layers

Layer	Input	Output
Flatten	tensor([3,32,32))	vector([x1, x2,...,x3072]); 3072 pixels
First Fully Connected: Linear(3 * 32 * 32, 256)	vector([x1, x2,...,x3072]); 3072 pixels	- Parameters: 3072*128 + 128= 393344 - 128 neurons
Second Fully Connected: Linear(128, 64)	- Vector[x1, x2, …, x128] - 128 neurons	- Parameters: 128*64 + 64 = 8256 - 128 neurons
Third Fully Connected: Linear(64, 10)	- Vector[x1, x2, …, x64] - 64 neurons	- Parameters: 64*10 + 10 = 650 - 10 neurons ~= 10 classes

2.1.2.2.1.2 MLP Forward Implementation

Order	Layer

1	**Flatten**
2	**First Fully Connected**
3	**BatchNorm1d**
4	**ReLU**
5	**Dropout (p=0.3)**
6	**Second Fully Connected**
7	**BatchNorm1d**
8	**ReLU**
9	**Dropout (p=0.2)**
10	**Third Fully Connected**

2.1.2.2.2 Analysis

The table of best MLP training result based on the below Hyperparameters:

Epoch	Learning Rate	Batch Size	Train Loss	Test Loss	Train Accuracy	Test Accuracy	Cross Entropy Function
150	0.0001	128	1.0416	1.2919	63.00%	54.00%	AdamW(weight_decay=1e-3)

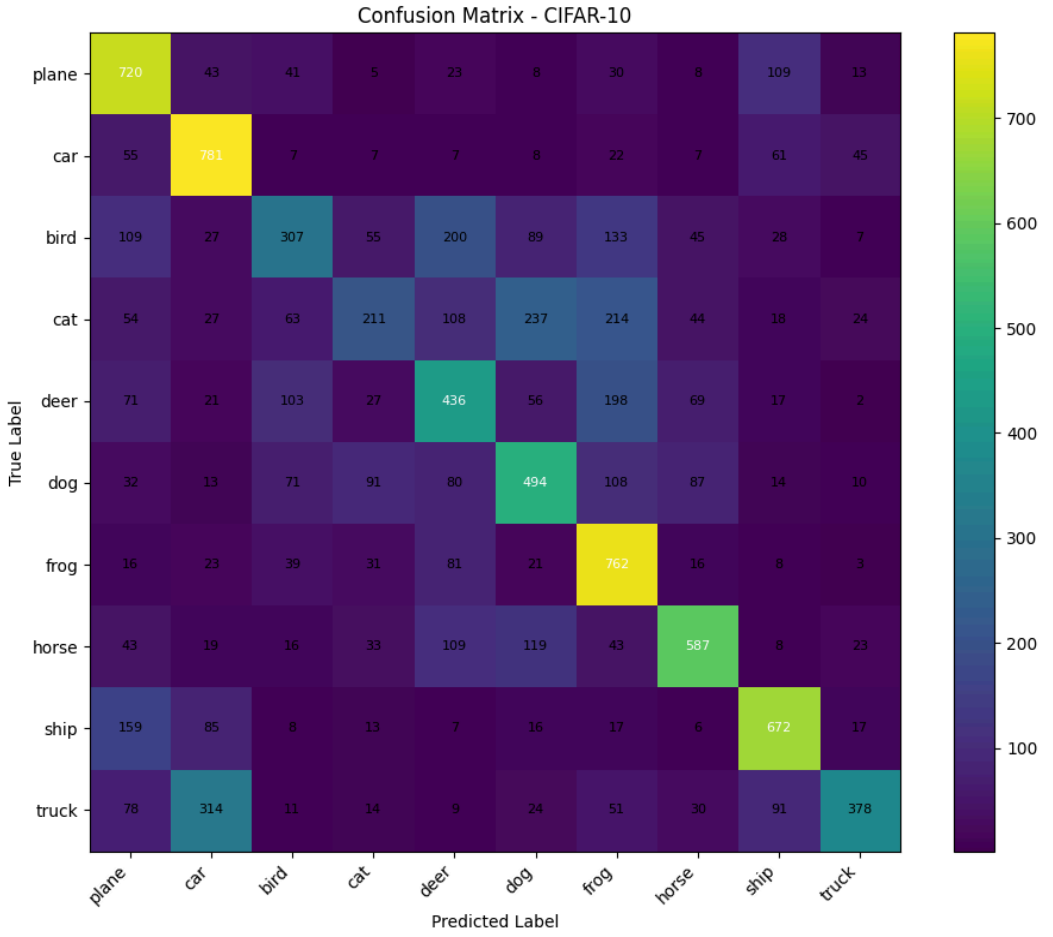


The training progress of MLP Model

The table accuracy of **10 classes** based on the **Test Dataset**

airplane	62.3%
automobile	66.3%
bird	37.8%
cat	33.8%
deer	47.8%
dog	43.0%
frog	63.0%
horse	60.09%
ship	69.3%
truck	60.3%

Confusion Matrix



The Confusion Matrix of MLP Model

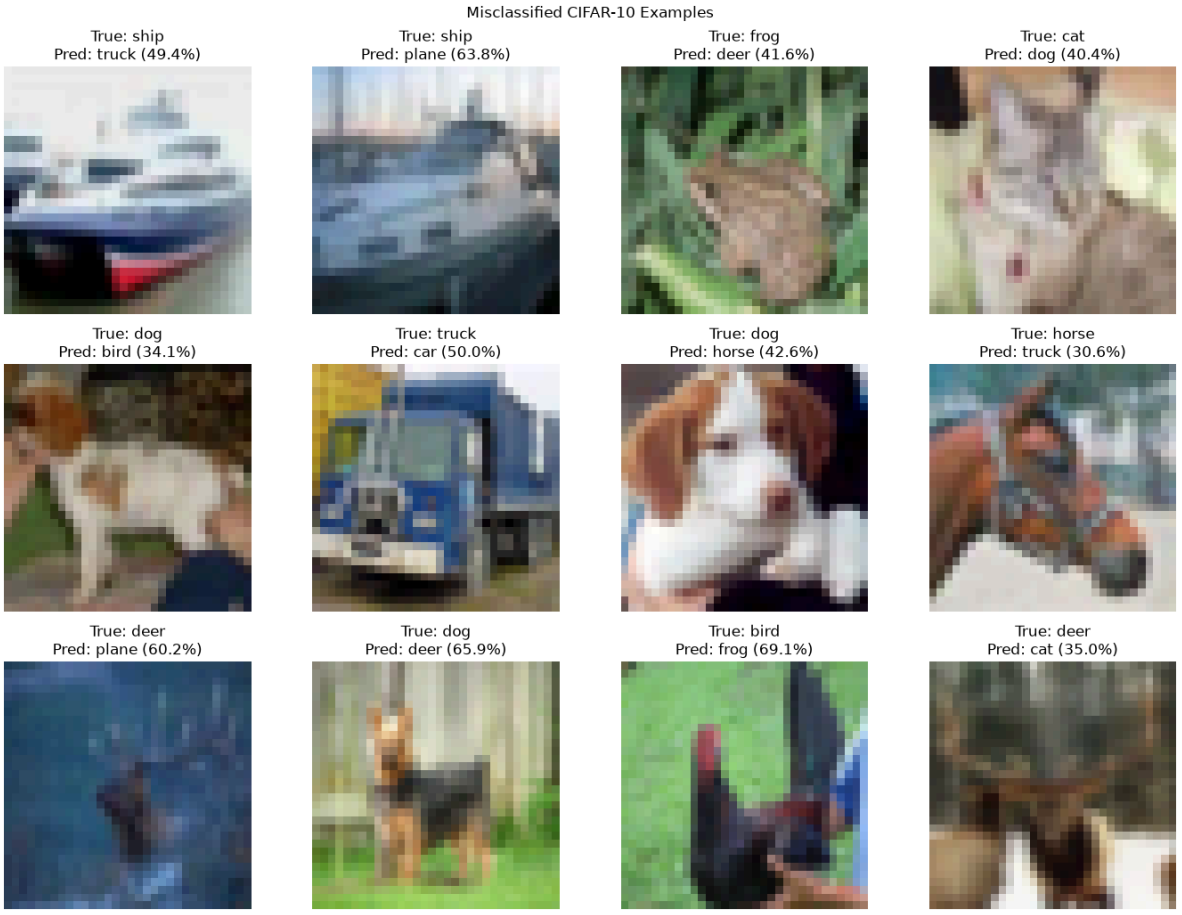
Classification Metric

classes	precision	recall	f1-score	samples
airplane	0.62	0.62	0.62	1000
automobile	0.63	0.66	0.65	1000
bird	0.46	0.38	0.41	1000
cat	0.39	0.34	0.36	1000
deer	0.46	0.48	0.47	1000
dog	0.43	0.43	0.43	1000
frog	0.59	0.63	0.61	1000
horse	0.59	0.61	0.60	1000
ship	0.64	0.69	0.67	1000
truck	0.58	0.60	0.59	1000

	precision	recall	f1-score	samples
accuracy			0.54	10000
macro avg	0.54	0.54	0.54	10000
weighted avg	0.54	0.54	0.54	10000

Display Misclassified Images based on Classes

Train Loss	Test Loss	Train Accuracy	Test Accuracy	Number of Test Errors
1.0416	1.2919	63.00%	54.00%	4555



The Misclassified CIFAR10 Examples of MLP Model

2.1.3 Convolutional Neural Network Classifier

2.1.3.1 Introduction

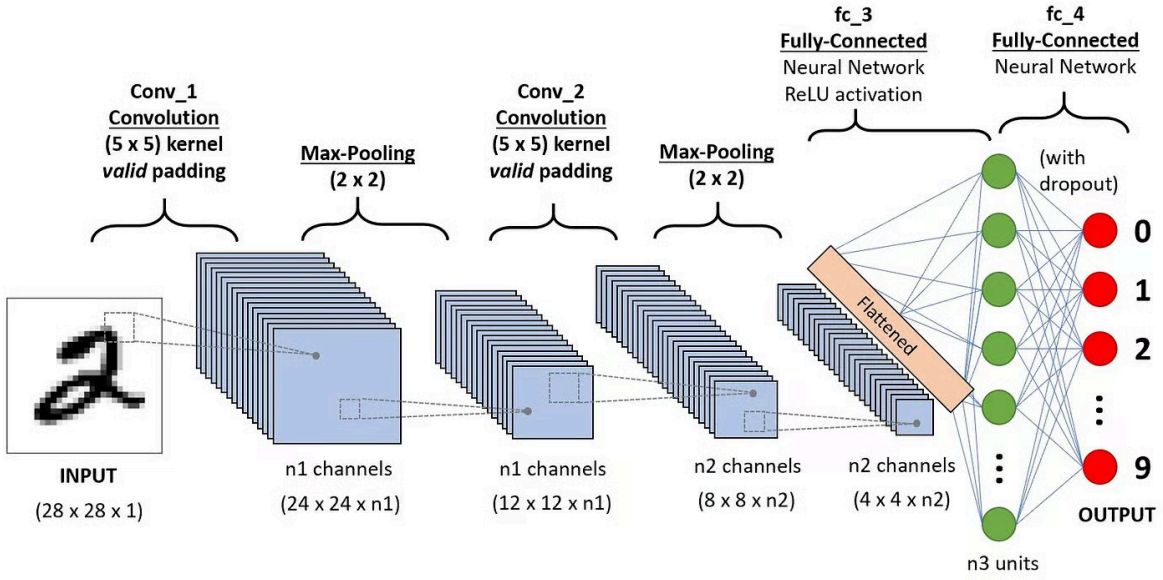
A Convolutional Neural Network (CNN) is a specialized type of deep learning algorithm mainly designed for tasks that necessitate object recognition, including image classification, detection, and segmentation. CNN is inspired by the human visual system, designed to process image data by capturing spatial relationships between pixels.
Convolutional neural networks were inspired by the layered architecture of the human visual cortex, and below are some key similarities and differences:

- **Hierarchical architecture:** Both CNNs and the visual cortex have a hierarchical structure, with simple features extracted in early layers and more complex features built up in deeper layers. This allows increasingly sophisticated representations of visual inputs.
- **Local connectivity:** Neurons in the visual cortex only connect to a local region of the input, not the entire visual field. Similarly, the neurons in a CNN layer are only connected to a local region of the input volume through the convolution operation. This local connectivity enables efficiency.

- **Translation invariance:** Visual cortex neurons can detect features regardless of their location in the visual field. Pooling layers in a CNN provide a degree of translation invariance by summarizing local features.
- **Multiple feature maps:** At each stage of visual processing, there are many different feature maps extracted. CNNs mimic this through multiple filter maps in each convolution layer.
- **Non-linearity:** Neurons in the visual cortex exhibit non-linear response properties. CNNs achieve non-linearity through activation functions like ReLU applied after each convolution.

The components of a CNN that help the CNNs mimic how the human brain operates to recognize patterns and features in images:

- Convolutional layers
- Rectified Linear Unit (ReLU for short)
- Pooling layers
- Fully connected layers



The CNN Architecture

2.1.3.2 Discussion And Experimental Analysis

2.1.3.2.1 Convolutional Neural Network Configuration

2.1.3.2.1.1 CNN Definition Layers

Layer	Input	Output

First Convolutional: Conv2d(in_channels = 3, out_channels = 6, kernel_size = 5)	- tensor([N,3,32,32])	- Parameters: 6*3*5*5 + 6 = 456 - tensor([N,6,28,28])
First Max Pooling: MaxPool2d(kernel_size = 2, stride = 2)	- tensor([N,6,28,28])	- tensor([N,6,14,14])
Second Convolutional: Conv2d(in_channels = 6, out_channels = 16, kernel_size = 5)	- tensor([N,6,14,14])	- Parameters: 16*6*5*5 + 16 = 2416 - tensor([N,16,10,10])
Second Max Pooling: MaxPool2d(kernel_size = 2, stride = 2)	- tensor([N,16,10,10])	- tensor([N,16,5,5])
First Fully Connected: Linear(16 * 5 * 5, 128)	- Vector[x1, x2, …, x400] - 400 features	- Parameters: 400*128 + 128 = 51328 - 128 neurons
Second Fully Connected: Linear(128, 64)	- Vector[x1, x2, …, x128] - 128 neurons	- Parameters: 128*64 + 64 = 8256 - 84 neurons
Third Fully Connected: Linear(64, 10)	- Vector[x1, x2, …, x10] - 64 neurons	- Parameters: 64*10 + 10 = 650 - 10 neurons ~= 10 classes

2.1.3.2.1.2 CNN Forward Implementation

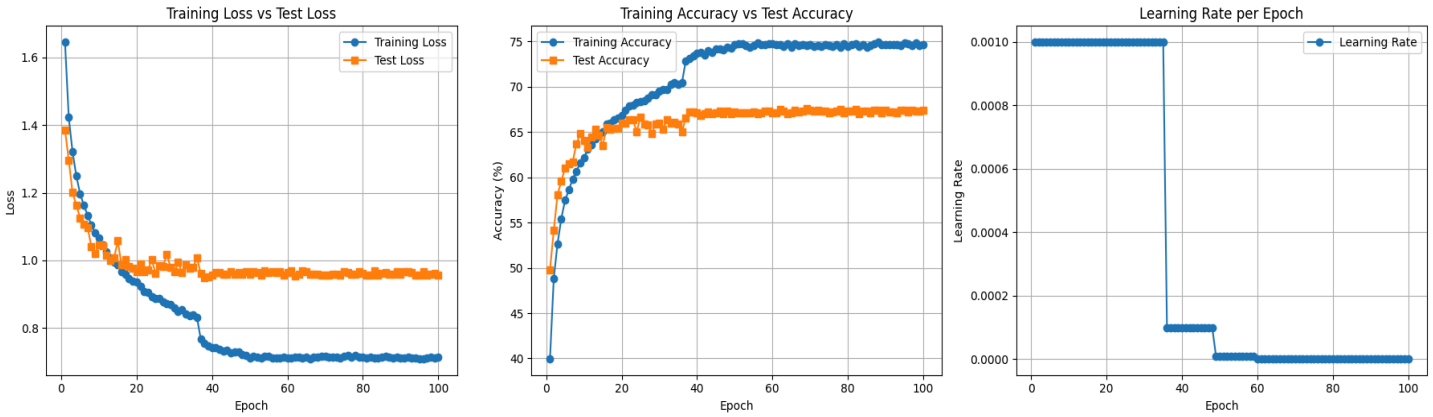
Order	Layer
1	**First Convolutional**
2	**BatchNorm1d**
3	**ReLU**
4	**Max Pooling**
5	**Second Convolutional**
6	**BatchNorm1d**
7	**ReLU**
8	**Max Pooling**
9	**Dropout (p**=0.25)
10	**Flatten**

11	**First Fully Connected**
12	**Second Fully Connected**
13	**Third Fully Connected**

2.1.3.2.2 Analysis

- **Hyperparameters Configuration 1 (Little Overfitting):**

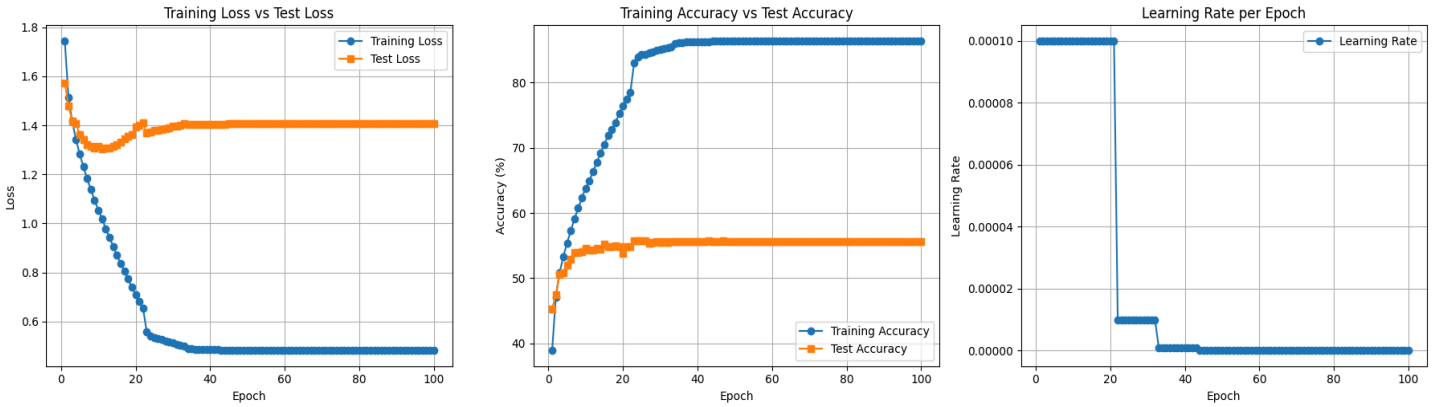
Epoch	Learning Rate	Batch Size	Train Loss	Test Loss	Train Accuracy	Test Accuracy	Cross Entropy Function
100	0.001	64	0.4	0.9	75%	67%	Adam



The training progress of CNN Model (1)

- **Hyperparameters Configuration 2 (Overfitting):**

Epoch	Learning Rate	Batch Size	Train Loss	Test Loss	Train Accuracy	Test Accuracy	Cross Entropy Function
100	0.0001	128	0.5	1.4	85%	55%	Adam



The training progress of CNN Model (2)

- **Hyperparameters Configuration 3 (best):**

Epoch	Learning Rate	Batch Size	Train Loss	Test Loss	Train Accuracy	Test Accuracy	Cross Entropy Function
150	0.0001	128	0.9542	0.9983	66.32	64.62	AdamW(weight_ decay=1e-3)

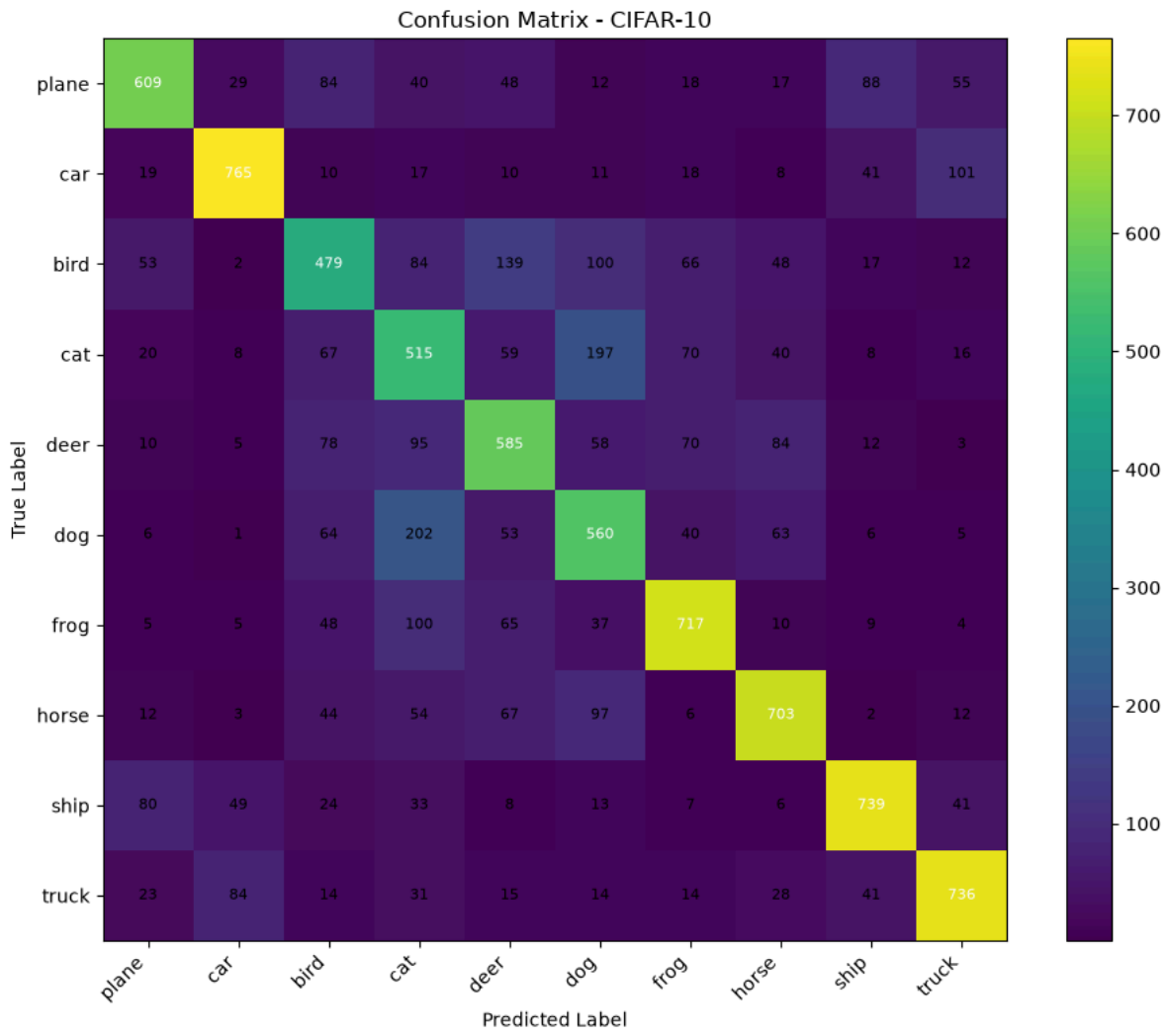


The training progress of CNN Model (best)

The table accuracy of **10 classes** based on the **Test Dataset**

airplane	**63.3%**
automobile	**73.9%**
bird	**46%**
cat	**39.9%**
deer	**52.0%**
dog	**47.7%**
frog	**71.0%**
horse	**66.3%**
ship	**72.6%**
truck	**71.7%**

Confusion Matrix



The Confusion Matrix of CNN Model

Classification Metric

classes	precision	recall	f1-score	samples
airplane	0.73	0.61	0.66	1000
automobile	0.80	0.77	0.78	1000
bird	0.53	0.48	0.50	1000
cat	0.44	0.52	0.47	1000
deer	0.56	0.58	0.57	1000
dog	0.51	0.56	0.53	1000
frog	0.70	0.72	0.71	1000
horse	0.70	0.70	0.70	1000

ship	0.77	0.74	0.75	1000
truck	0.75	0.74	0.74	1000

	precision	**recall**	**f1-score**	**samples**
accuracy			0.64	10000
macro avg	0.65	0.64	0.64	10000
weighted avg	0.65	0.64	0.64	10000

Display Misclassified Images based on Classes:



The Misclassified CIFAR10 Examples of CNN Model

2.1.4 The Comparison And Evaluation

2.1.4.1 Learning Curve

	Learning Curve
CNN	**Loss Dynamics**: Both training loss (from 2.04 to 0.95) and test loss (from 1.77 to 0.99) decrease steadily and remain tightly aligned throughout all 150 epochs. **Accuracy Dynamics**: Training accuracy reaches **66.5%**, while test accuracy reaches **64.7%**. **Generalization & Fitting**: Exhibits the strongest generalization capability among the three models with negligible overfitting, as evidenced by the minimal gap (1.8%) between training and test curves. Convolutional layers effectively capture local spatial hierarchies and translational invariant patterns in visual data
MLP	**Loss Dynamics**: Training loss continues decreasing down to 1.04, whereas test loss plateaus near 1.29 around epoch 35–40. **Accuracy Dynamics**: Training accuracy climbs to **~63.0%**, but test accuracy plateaus early at **54.0%**. **Generalization & Fitting**: Demonstrates clear **overfitting** starting around epoch 30. Dense fully connected layers learn memorized pixel combinations rather than spatial features, leading to a noticeable performance gap (9.0%) between training and test performance.
Softmax	**Loss Dynamics**: High overall residual loss. Training loss plateaus around 1.62 and test loss around 1.71. **Accuracy Dynamics**: Training accuracy caps at **45.3%**, with test accuracy leveling off at **40.3%**. **Generalization & Fitting**: Suffers from significant **underfitting** due to limited model capacity. A linear model using Softmax without hidden layers or non-linear spatial transformations cannot learn complex decision boundaries required for raw pixel inputs.

2.1.4.1 Classification Metric

	Classification Metric
CNN	**Overall Assessment**: Superior performance across all metrics with an F1-score of **0.6430** and Accuracy of **64.08%**. **Precision/Recall Balance**: Precision (0.6476) slightly exceeds recall (0.6408), indicating high confidence when predicting positive class labels with minimal false positives.
MLP	**Overall Assessment**: Moderate performance with an F1-score of **0.5411** and Accuracy of **54.45%**. **Precision/Recall Balance**: Balanced precision (0.5393) and recall (0.5445) reflect balanced multi-class prediction, though limited in overall discriminatory strength.

Softmax	**Overall Assessment**: Lowest overall capacity with an F1-score of **0.4103** and Accuracy of **41.41%**. **Precision/Recall Balance**: Low precision (0.4090) and recall (0.4141) demonstrate high structural bias and underfitting.

2.2 E2

2.2.1 Introduction

The remarkable success of the Transformer architecture in Natural Language Processing has strongly inspired its adoption in computer vision, giving rise to the Vision Transformer (ViT) model. Unlike Convolutional Neural Networks (CNNs), which rely heavily on localized receptive fields and translation invariance, Transformers process images through the lens of the Self-Attention mechanism. This core mechanism enables models to simultaneously capture global dependencies across disparate spatial regions of an image, irrespective of their physical distance.

However, adapting Transformers to visual data presents two primary challenges that must be addressed:

1. Tokenization (Image-to-Sequence Transformation): Unlike text, images lack an inherent sequential structure. Therefore, effective strategies are required to segment and map spatial image representations into discrete tokens (e.g., partitioning into grid patches or treating individual pixels as tokens).
2. Multi-Head Self-Attention (MSA): This mechanism allows the model to jointly attend to information from different representation subspaces at various positions, enabling deep extraction of complex contextual visual features.

2.2.2 Multi-Head Self-Attention (MSA)

The core Self-Attention mechanism operates on three component matrices—Query (Q), Key (K), and Value (V)—projected linearly from the input representation $X \in R^{B\times N\times D}$. The mathematical formulation of Scaled Dot-Product Attention is defined as:

$$Attention(Q,K,V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Q and K have dimensions (B, H, N, d_k), with H denoting the number of attention heads and $d_k = D/H$ representing the dimension of each head.

The matrix multiplication QK^T computes the similarity scores (attention weights) between tokens across the sequence.

The scaling factor $\sqrt{d_k}$ is critical to prevent excessively large values of inner products when d_k is high, which would otherwise push the softmax function into regions with extremely small gradients (vanishing gradients).

In Multi-Head Attention, this attention function is executed in parallel across H different subspaces, and the resulting outputs are concatenated and linearly projected via an output weight matrix W_o.

When adapting or "transforming" a Vision Transformer to work effectively on small datasets (such as CIFAR-10 or Tiny-ImageNet), standard ViTs usually struggle because they lack

locality inductive bias (the built-in assumption that neighboring pixels are related, which CNNs naturally have).
To fix this limitation when training ViT-Small on small datasets, researchers and engineers apply specific architectural modifications and transformation pipelines.

2.2.3 ViT with Custom MSA Definition Layers (Patch Token 4x4)

Layer	Input Shape	Output Shape & Parameters
PatchTokenization: Linear(48, 64)	tensor([B, 3, 32, 32])	- Output: tensor([B, 64, 64]) (64 tokens) - Parameters: 48 * 64 + 64 = 3,136
Positional Embedding: nn.Parameter	tensor([B, 64, 64])	- Output: tensor([B, 64, 64]) - Parameters: 1 * 64 * 64 = 4,096 (Learnable)
Transformer Encoder Block:		
1. norm1: LayerNorm(64)	tensor([B, 64, 64])	- Output: tensor([B, 64, 64]) - Parameters: 64 * 2 = 128
2. CustomMSA (embed_dim=64, num_heads=4): - q_proj: Linear(64, 64) - k_proj: Linear(64, 64) - v_proj: Linear(64, 64) - out_proj: Linear(64, 64)	tensor([B, 64, 64])	- Parameters: 64 * 64 + 64 = 4,160 each (Total: 16,640)
3. norm2: LayerNorm(64)	tensor([B, 64, 64])	- Output: tensor([B, 64, 64]) - Parameters: 64 * 2 = 128
4. FFN (Sequential): Linear(64, 128) -> GELU -> Dropout -> Linear(128, 64)	tensor([B, 64, 64])	- Parameters: 64 * 128 + 128 = 8,320 (Linear 1) + 128 * 64 + 64 = 8,256 (Linear 2) (Total: 16,576)
Classification Head: - norm: LayerNorm(64) - Global Average Pooling (mean(dim=1)) - head: Linear(64, 10)	tensor([B, 64, 64])	- Output: tensor([B, 10]) (10 classes) - Parameters: LayerNorm = 128, Linear = 650

2.2.4 ViT with Custom MSA Forward Implementation Flow

Order	Operation / Layer	Description / Transformation

1	**Patch Tokenization**	Converts image tensor (B, 3, 32, 32) into token sequence (B, 64, 64)
2	**Positional Embedding Addition**	Adds learnable 1D spatial positional embeddings directly to token representations
3	**First LayerNorm (norm1)**	Pre-Layer Normalization before passing tokens into Custom MSA (Pre-LN architecture)
4	**Custom MSA Projection (Q, K, V)**	Linear projection of normalized tokens into Query, Key, and Value subspaces
5	**Multi-Head Split & Transpose**	Splits embedding dimension into 4 heads, reshaping tensor to (B, 4, 64, 16)
6	**Scaled Dot-Product Attention**	Computes attention map: softmax((Q * K^T) / sqrt(d_k)) * V
7	**Concat Heads & Output Projection**	Concatenates multi-head outputs and projects via out_proj back to (B, 64, 64)
8	**Residual Connection 1**	Adds shortcut connection: x = x + MSA(norm1(x))
9	**Second LayerNorm (norm2)**	Pre-Layer Normalization before passing tokens into the Feed-Forward Network
10	**Feed-Forward Network (FFN)**	Processes tokens via Linear(64, 128) -> GELU -> Dropout -> Linear(128, 64)
11	**Residual Connection 2**	Adds shortcut connection: x = x + FFN(norm2(x))
12	**Final LayerNorm (norm)**	Final layer normalization before classification head projection
13	**Global Average Pooling (mean(dim=1))**	Aggregates token sequence (B, 64, 64) along sequence length into single feature vector (B, 64)
14	**Classification Head (head)**	Linear projection from 64-dimensional feature vector to 10 class logits (Linear(64, 10))

2.2.5 Hyperparameters Configuration and Architectural Justifications

Hyperparameter	Value	Description / Scope
Dataset	CIFAR-10	32x32x3 RGB images, 10 classes
Train Samples	10,000	Subset extracted from CIFAR-10 training set
Test Samples	2,000	Subset extracted from CIFAR-10 test set
Batch Size	64	Number of samples processed per iteration

Epochs	20	Total training passes over the dataset
Learning Rate (lr)	1e-3 (0.001)	Initial step size for parameter updates
Optimizer	AdamW	Optimization algorithm with decoupled weight decay
Loss Function	CrossEntropyLoss	Objective function for multi-class classification
Embedding Dimension (D)	64	Dimensionality of projected token vectors
Number of Heads (H)	4	Parallel attention heads (head_dim = 16)
Patch Size (p)	4x4	Patch dimension for Patch Tokenization (N = 64 tokens)
Pixel Microbatch Size	8	Sub-batching parameter to mitigate VRAM bottlenecks for Pixel Tokenization (N = 1024 tokens)

Purpose for Selected Hyperparameters:

- **Train/Test Subsets (10,000 / 2,000):** Speeds up training cycles on Google Colab hardware while preserving statistical validity for model performance evaluation.
- **Batch Size (128):** Maximizes GPU parallel computing throughput without triggering memory overflow.
- **Epochs (20):** Provides adequate iterations for model convergence and performance divergence across configurations.
- **Optimizer (AdamW, lr = 1e-3):** Ensures stable gradient updates and effective weight decay regularization for Transformer attention layers.
- **Embedding Dimension (D = 64) & Heads (H = 4):** Establishes a compact Vision Transformer architecture with a head dimension of 16 (d_k = 64/4), balancing feature extraction capacity and computational efficiency.
- **Patch Size (4x4):** Yields 64 tokens per image, providing a spatial representation that balances local feature extraction with low attention complexity O(N^2).
- **Pixel Microbatch Size (8):** Mitigates memory bottlenecks caused by the 1024-token sequence in Pixel Tokenization, preventing Out-Of-Memory errors during backpropagation.

2.2.6 MSA Implementation

Most of the functional operations are embedded directly within the PyTorch model classes, where the __init__ method initializes the necessary layers and learnable parameters, and the forward method dictates the sequential tensor transformation flow.

Architectural Classes and Functional Components:

- **CustomMSA**:
    - **__init__**: Instantiates linear projection modules for Query (Q), Key (K), and Value (V), along with the output linear projection layer, Softmax activation, and Dropout layer.

    - **forward**: Computes the Multi-Head Self-Attention from scratch by projecting inputs into Q, K, and V matrices, splitting them across spatial heads, calculating pairwise token attention scores, and recombining multi-head feature representations.
- **PyTorchMSA**:
    - Uses PyTorch's native nn.MultiheadAttention module as a replacement for the manual MSA implementation.
    - **forward**: Passes the input tensor x simultaneously as query, key, and value to calculate self-attention.
- **PatchTokenization**:
    - Partitions a 32 × 32 CIFAR-10 image into non-overlapping 4 × 4 spatial patches.
    - Generates a sequence of 64 tokens per image, projecting each patch into a dense vector of dimension embed_dim.
- **PixelTokenization**:
    - Treats each individual pixel as an independent sequence token.
    - Produces 1,024 tokens for a 32 × 32 image, resulting in substantially higher computational complexity during attention matrix calculations compared to patch-based tokenization.
- **TransformerEncoderBlock**:
    - Dynamically selects between CustomMSA and PyTorchMSA based on the specified msa_type.
    - Applies a Pre-Layer Normalization architecture with residual connections around both the MSA block and the Feed-Forward Network (FFN).
- **SmallViTClassifier**:
    - Integrates the tokenization module, learnable 1D spatial positional embeddings, Transformer encoder blocks, and the classification head.
    - **forward**: Maps raw images to token sequences, adds positional embeddings, passes them through encoder blocks, applies Global Average Pooling across sequence tokens, and outputs predictions across 10 target classes.
- **train_and_evaluate(...)**:
    - Handles model optimization using Cross-Entropy Loss and the AdamW optimizer, logging epoch and batch progress.
    - Implements a microbatch_size parameter to split large batches during forward passes, mitigating GPU VRAM bottlenecks during pixel-based tokenization experiments.
    - Evaluates performance on the test set post-training to generate the target confusion matrix.
    - Returns the final accuracy, total training wall-clock time, and test confusion matrix.

Data Pipeline and Experimental Execution Flow

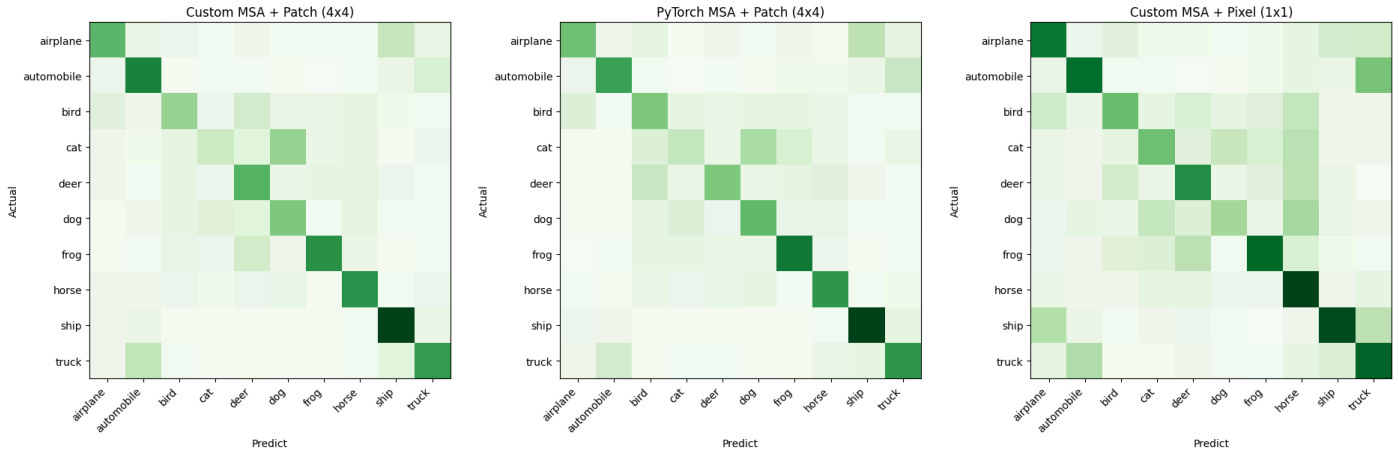
- **Data Preparation**: Utilizes torchvision.transforms to normalize image distributions and convert images into PyTorch tensors. Subset constrains the dataset size, while DataLoader manages mini-batch iteration.

- **Experimentation Suite**: Executes three experimental configurations across Custom vs. PyTorch MSA and Patch vs. Pixel tokenization strategies, invoking train_and_evaluate and logging performance artifacts into a central results structure.
- **Evaluation Metrics Computation**: Extracts confusion matrices from results to calculate macro-averaged Precision, Recall, and F1-score across classes without needing to retrain models.

2.2.7 Result Analyzing

To evaluate the empirical behavior of Vision Transformers on small-scale image datasets, three distinct model configurations were benchmarked on the CIFAR-10 classification task. The experimental results, including classification metrics and total training duration (wall-clock time), are summarized in the table below:

Configuration	Accuracy (%)	Wall-Clock Time (s)	Precision (Macro)	Recall (Macro)	F1-Score (Macro)
Custom MSA + Patch Tokenization (4x4)	**52.85**	71.67	**0.5183**	**0.5249**	**0.5156**
PyTorch MSA + Patch Tokenization (4x4)	51.65	**69.83**	0.5101	0.5128	0.5068
Custom MSA + Pixel Tokenization (1x1)	44.90	489.06	0.4510	0.4466	0.4412



1. Patch-Based (4x4) vs. Pixel-Based (1x1) Tokenization

- Classification Performance: Patch-based tokenization (4 × 4) demonstrates a clear advantage over pixel-based tokenization (1 × 1), achieving a superior accuracy of 52.85% compared to 44.90% (an absolute gain of +7.95%), along with a corresponding macro F1-score improvement from 0.4412 to 0.5156.

- Computational Efficiency & Execution Time: The training duration exhibits a massive disparity. Patch tokenization completes 20 training epochs in approximately 71.67 seconds, whereas pixel tokenization requires 489.06 seconds (a ≈ 6.8× slowdown), even after employing a reduced PIXEL_MICROBATCH_SIZE = 8 to mitigate GPU VRAM overflow.
- Architectural Justification:
    - For a 32 × 32 CIFAR-10 image, pixel-level tokenization treats each pixel as a standalone token, creating a long sequence length of N = 1,024 tokens. Because the Self-Attention mechanism scales quadratically with sequence length (O(N^2)), computing attention matrices across 1,024 tokens imposes a severe computational and memory bottleneck.
    - Furthermore, individual pixels lack local semantic context. In contrast, patch-based tokenization (4 × 4) aggregates local pixel neighborhoods into N = 64 sequence tokens. This spatial grouping introduces an implicit inductive spatial structure while dramatically lowering self-attention complexity, allowing the model to learn meaningful visual representations far more effectively.

2. Custom MSA vs. PyTorch Native MSA (Patch 4x4)

- Predictive Accuracy: The custom Multi-Head Self-Attention (CustomMSA) implementation achieves slightly higher classification performance (52.85% accuracy, 0.5156 F1-score) than PyTorch's native nn.MultiheadAttention (51.65% accuracy, 0.5068 F1-score).
- Training Throughput: PyTorch's native MSA executes faster (69.83s vs. 71.67s), benefiting from highly optimized C++/CUDA CUDNN kernels under the hood.
- Architectural Justification: Since both implementations share the identical sequence length (N = 64 tokens) and head dimension (d_k = 16), their underlying mathematical self-attention formulation is functionally equivalent. The minor accuracy margin (≈ 1.2%) can be attributed to subtle differences in parameter weight initialization, numerical precision during matrix operations, or dropout masking behavior.

Summary:

1. **Patch Tokenization is Essential for ViTs:** Grouping pixels into local patches (4 × 4) is vital for Vision Transformers, as it simultaneously provides rich spatial semantics and mitigates the quadratic computational bottleneck (O(N^2)) of full self-attention.
2. **Pixel-Level Tokenization Bottleneck:** Treating images as 1D sequences of individual pixels (1 × 1) leads to poor feature representations and extreme computational inefficiency.

2.3 E3

2.3.1 Long Short-Term Memory (LSTM)

2.3.1.1 Introduction

A Long Short-Term Memory (LSTM) network is a specialized type of Recurrent Neural Network (RNN) designed to learn long-term dependencies in sequential data, such as time series or text. Unlike standard RNNs, which suffer from the vanishing gradient problem, LSTMs use a conveyor belt-like cell state and a system of gates to control the flow of information.

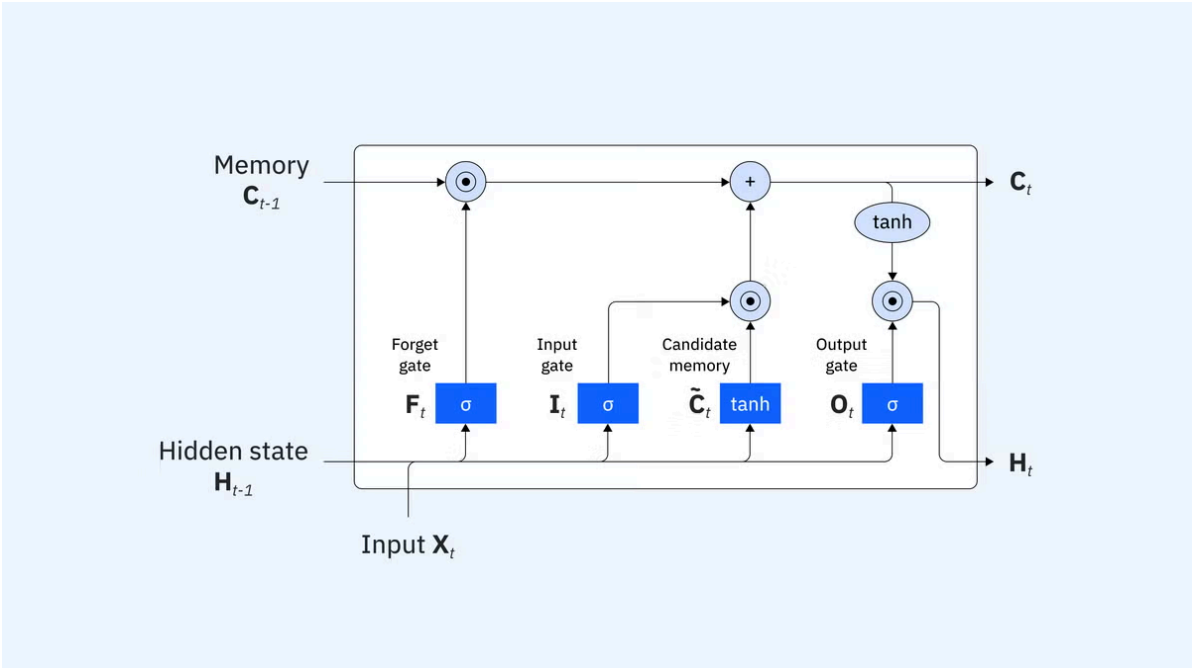
How does the LSTM solve the RNN problem (Vanishing Gradient)?

LSTMs add a separate memory track, called the cell state, that runs in parallel to the standard hidden state. The difference is structural: information in the cell state flows through addition rather than repeated multiplication, so it doesn't degrade the same way.

To control what goes into and out of this memory, LSTMs use gates. Each gate is a small neural network that learns to output values between 0 and 1, acting as a filter. A value near 0 blocks information; a value near 1 lets it through. The network learns during training which patterns are worth remembering across long sequences and which can be discarded.

This gating mechanism is what separates LSTMs from vanilla RNNs. Instead of forcing all information through the same path, LSTMs can selectively preserve context across hundreds of timesteps. The next section breaks down exactly how these gates work and how they combine to update the cell state.

The LSTM Architecture



The LSTM Architecture

LSTMs are similar to many other kinds of feed-forward neural networks but they have several crucial differences. The LSTM model introduces an intermediate type of storage with a structure that's called a memory cell. A memory cell is a composite kind of unit built from simpler nodes, the most important of which is a multiplicative node.

The multiplicative nodes in an LSTM memory cell function like gates that control the flow of information through the network. They enable the network to decide how much of each signal from the input data should pass through. There are three main places where these multiplicative interactions occur. An LSTM cell contains three gates and one cell state.

The **forget gate** multiplies the previous cell state by another value between 0 and 1, deciding how much information from the previous time step to retain or discard.

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

The **input gate** multiplies the candidate cell state by a value produced by a sigmoid function as the activation function, between 0 and 1, determining how much new information from the current input to add to the memory.

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

Finally, the **output gate** multiplies the updated cell state (after passing through a tanh) by a gating value to determine what part of the internal state becomes visible as output ot and a new hidden state ht :

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$\tilde{C}_t = tanh(W_c[h_{t-1}, x_t] + b_c)$$

The multiplicative operations at each gate enable the LSTM to regulate information dynamically. That protects long-term dependencies from being overwritten or vanishing.

2.3.1.2 Discussion And Experimental Analysis

2.3.1.2.1 Long Short-Term Memory (LSTM) Configuration

2.3.1.2.1.1 LSTM Definition Layers

Layer	Input	Output
Long Short-Term Memory: LSTM(input_size,hidden_size,num_layers,batch_first,dropout)	- tensor([N,32,96])	- tensor([N,32,hidden_size])
Dropout: Dropout(p=0.2)		
First Fully Connected: Linear(hidden_size, num_classes = 10)	- tensor([N,hidded_size])	- tensor([N,10])

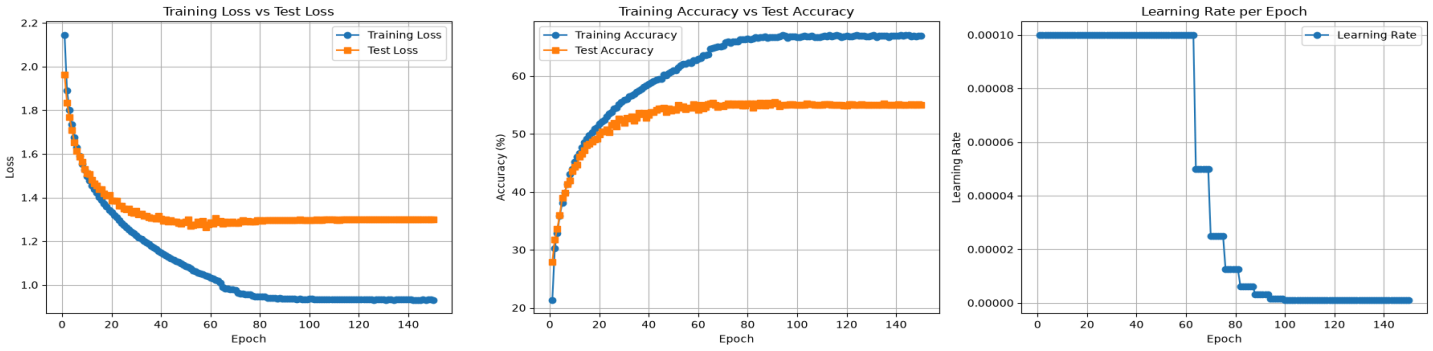
2.3.1.2.1.2 LSTM Forward Implementation

Order	Layer
1	**Convert initial tensor = ([B,C,H,W]) to tensor = ([B,Sequences,Feature Maps])** (tensor = ([128,3,32,32]) ⇒ tensor = ([128,32,96]))
2	**LSTM Layer** **output, (h_n, c_n) = self.lstm(x)** (**x** is input tensor ([B,Sequences,Feature Maps])) (**output:** Contains the hidden states for all time steps in the sequence for the final LSTM layer. At each row (from 1 to 32), the LSTM outputs a hidden vector representing the information accumulated up to that row. The output stores all 32 of these hidden vectors. Shape is **[B, Sequences, Hidden_Size]**) (**h_n:** Final hidden state**.** The hidden state (short-term memory) at the final time step—corresponding to the 32nd row, the last row in the image—across all LSTM layers. It encapsulates the complete information from all 32 rows after the LSTM has finished processing the sequence from top to bottom.Shape is **[number_layers * num_directions, B, hidden_size]**) (**c_n**: Final Cell state. The cell state (long-term memory) at the final time step (index 32) across all LSTM layers. Used to maintain the flow of information across time steps during the backpropagation algorithm. Shape is **[number_layers * num_directions, B, hidden_size]**)
3	**h_n[-1]** (Get the hidden state of last step)
4	**Dropout (p=h_n[-1])**
5	**Fully Connected**

2.3.1.2.2 Analysis

- **Hyperparameters Configuration:**

Epoch	Learning Rate	Batch Size	Train Loss	Test Loss	Train Accuracy	Test Accuracy	Cross Entropy Function
150	0.0001	128	0.9298	1.3001	66.88%	55.04%	AdamW(weight_ decay=0.001)

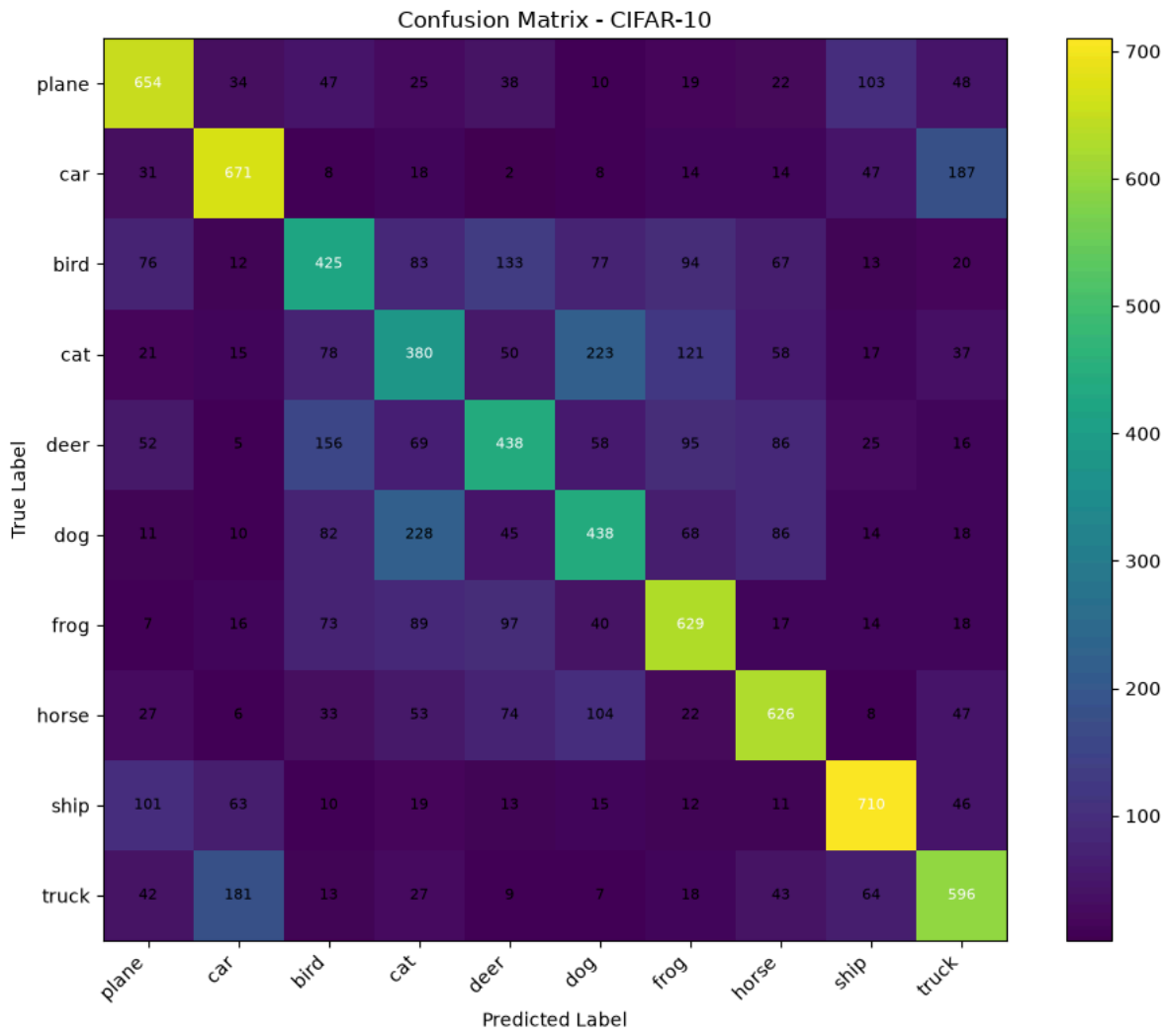


The training progress of LSTM Model

The table accuracy of **10 classes** based on the **Test Dataset**

airplane	**64.5%**
automobile	**66.6%**
bird	**42.0%**
cat	**38.1%**
deer	**45.6%**
dog	**44.0%**
frog	**62.9%**
horse	**61.6%**
ship	**70.5%**
truck	**59.7%**

Confusion Matrix



The Confusion Matrix of LSTM Model

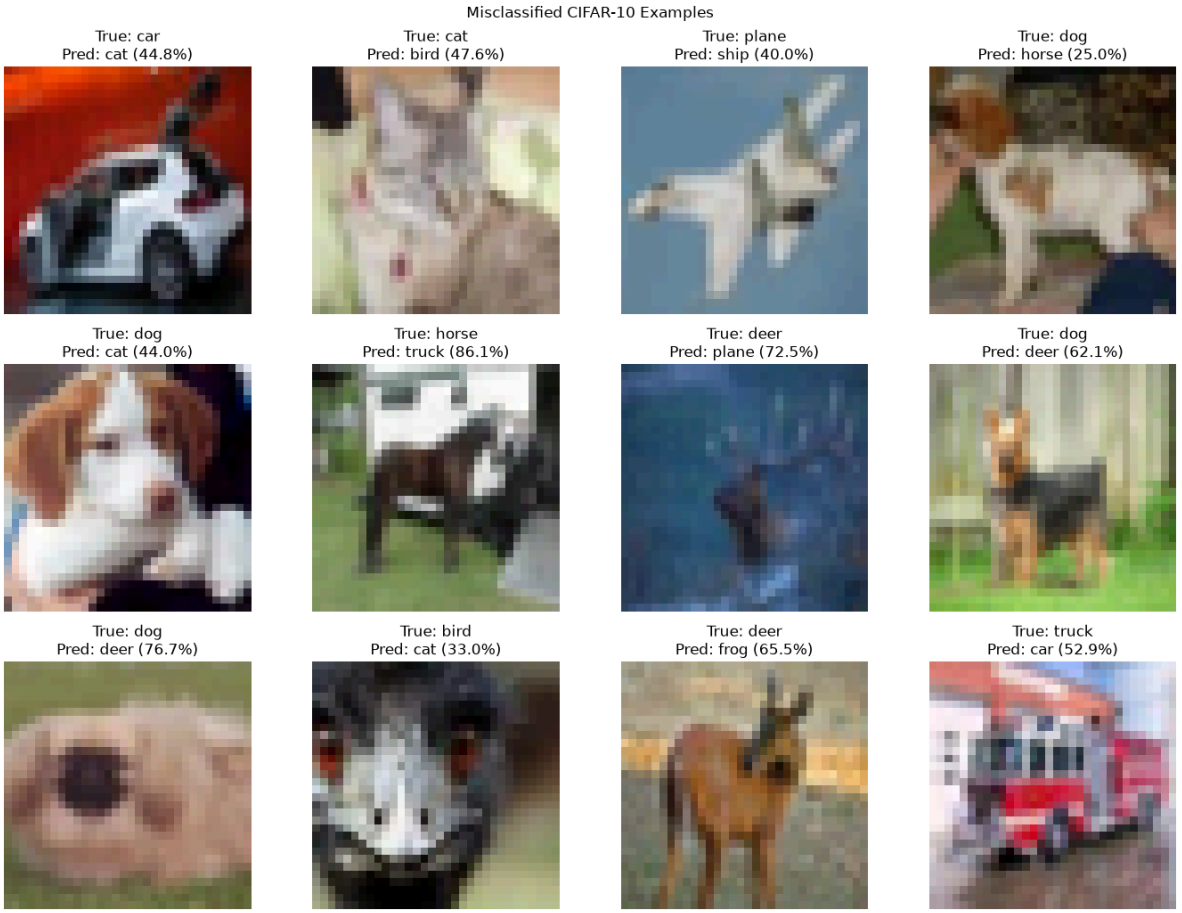
Classification Metric

classes	precision	recall	f1-score	samples
airplane	0.64	0.65	0.66	1000
automobile	0.66	0.77	0.78	1000
bird	0.46	0.48	0.50	1000
cat	0.38	0.52	0.47	1000
deer	0.49	0.58	0.57	1000
dog	0.45	0.56	0.53	1000
frog	0.58	0.72	0.71	1000
horse	0.61	0.70	0.70	1000

ship	0.70	0.74	0.75	1000
truck	0.58	0.74	0.74	1000

	precision	**recall**	**f1-score**	**samples**
accuracy			0.56	10000
macro avg	0.55	0.56	0.55	10000
weighted avg	0.55	0.56	0.55	10000

Display Misclassified Images based on Classes:



The Misclassified CIFAR10 Examples of LSTM Model

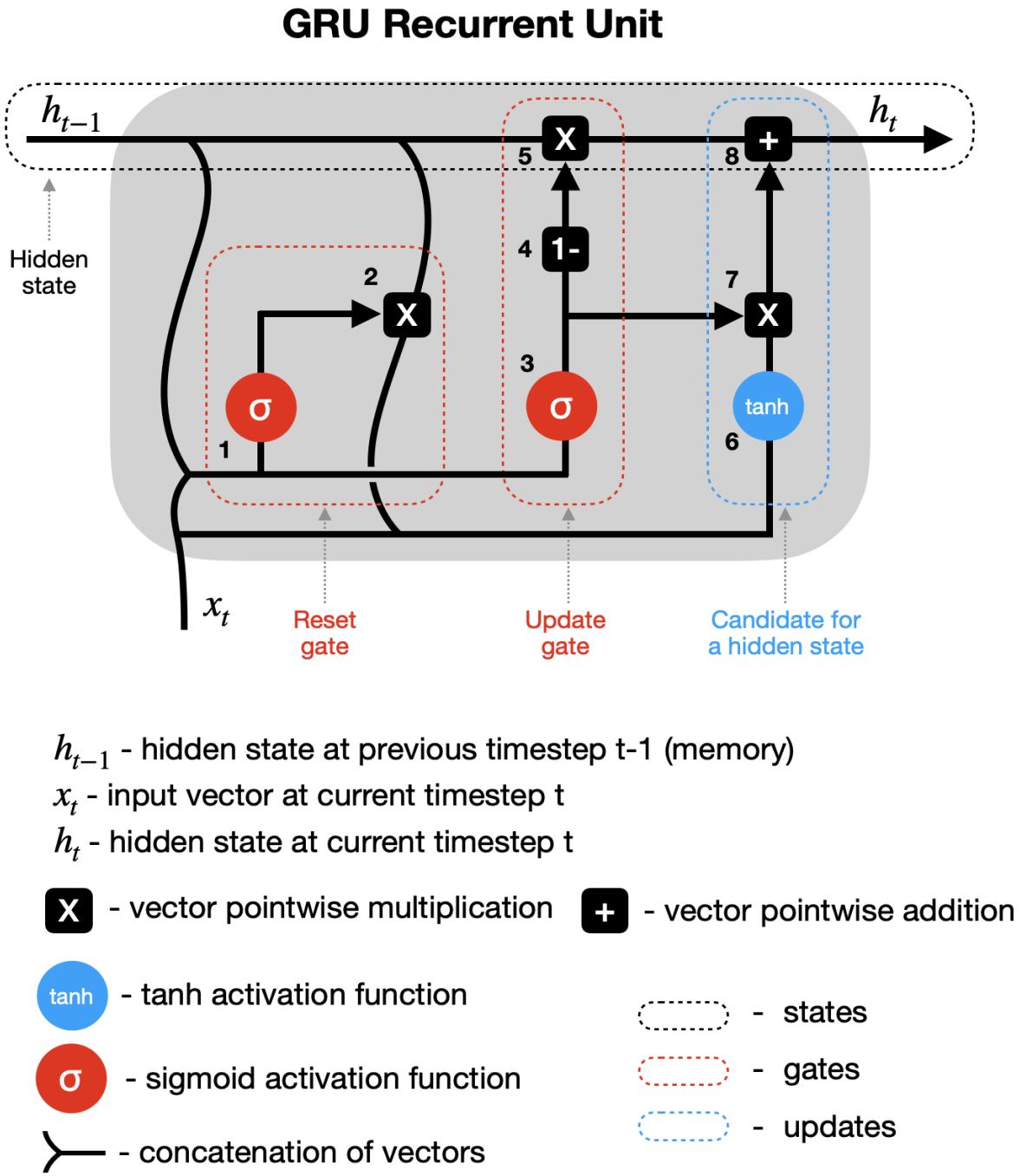
2.3.2 Gated Recurrent Unit (GRU)

2.3.2.1 Introduction

Recurrent Neural Networks (RNNs) have proven to be a great solution for processing sequential data. But RNNs are often challenged by a problem known as the **vanishing**

gradient problem, which makes them struggle with learning long-term dependencies. The **GRU (Gated Recurrent Unit)** was introduced as a simpler and more efficient alternative to the LSTM (Long Short-Term Memory) network to handle this issue.

The GRU Architecture



The GRU Architecture

Reset gate

Previous hidden state (h_t-1) and current input (x_t) are combined (multiplied by their respective weights and bias added) and passed through a reset gate. Since the sigmoid function ranges between 0 and 1, step one sets which values should be discarded (0), remembered (1), or partially retained (between 0 and 1). Step two resets the previous hidden state multiplying it with outputs from step one.

Mathematical Formula:

$r_t = \sigma (W_r \cdot [h_{t-1} , x_t] + b_r)$

Where:

- r_t is the reset gate value at time step t.
- W_r is the reset gate weight.
- h_{t-1} is the hidden state from the previous time step.
- x_t is the current input.
- b_r is the bias term.
- σ is the sigmoid activation function that outputs a value between 0 and 1.
- A value of 0 means the model forgets everything from the previous hidden state, while a value of 1 means it keeps all the information.

Update Gate

The update gate decides how much of the new information should be passed to the current hidden state and how much of the previous state should be retained. This gate essentially controls how much memory from the past should be kept and how much should be updated with the new input.

Mathematical Formula:

$z_t = \sigma (W_z \cdot [h_{t-1} , x_t] + b_z)$

Where:

- z_t is the update gate value at time step t.
- W_z is the weight for the update gate.
- h_{t-1} is the previous hidden state.
- x_t is the current input.
- b_z is the bias term.
- The output of this gate is also between 0 and 1. A value of 0 means the model relies entirely on the previous hidden state, and a value of 1 means it relies entirely on the new input.

2.3.2.2 Discussion And Experimental Analysis

2.3.2.2.1 Gated Recurrent Unit Configuration

2.3.2.2.1.1 GRU Definition Layers

Layer	Input	Output
Long Short-Term Memory: LSTM(input_size,hidden_size,num_layers,batch_first,dropout)	- tensor([N,32,96])	- tensor([N,32,hidden_size])
Dropout: Dropout(p=0.2)		
First Fully Connected:	- tensor([N,hidded_size])	- tensor([N,10])

Linear(hidden_size, num_classes = 10)		

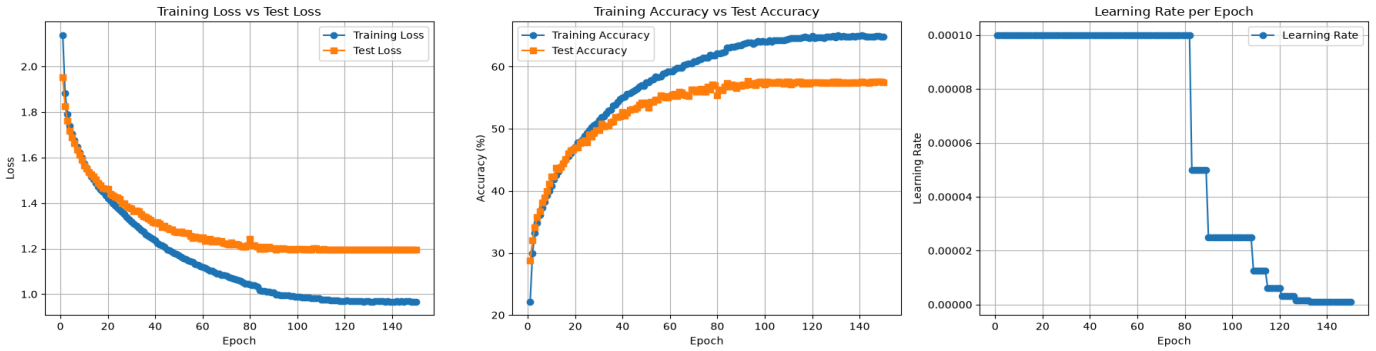
2.3.2.2.1.2 GRU Forward Implementation

Order	Layer
1	**Convert initial tensor = ([B,C,H,W]) to tensor = ([B,Sequences,Feature Maps])** (tensor = ([128,3,32,32]) ⇒ tensor = ([128,32,96]))
2	**LSTM Layer** **output, h_n = self.gru(x)** (**x** is input tensor ([B,Sequences,Feature Maps])) (**output:** Contains the hidden states for all time steps in the sequence for the final GRU layer. At each row (from 1 to 32), the GRU outputs a hidden vector representing the information accumulated up to that row. The output stores all 32 of these hidden vectors. Shape is **[B, Sequences, Hidden_Size]**) (**h_n:** Final hidden state**.** The hidden state at the final time step—corresponding to the 32nd row, the last row in the image—across all GRU layers. It encapsulates the complete information from all 32 rows after the GRU has finished processing the sequence from top to bottom.Shape is **[number_layers * num_directions, B, hidden_size]**)
3	**h_n[-1]** (Get the hidden state of last step)
4	**Dropout (p=h_n[-1])**
5	**Fully Connected**

2.3.2.2.2 Analysis

- **Hyperparameters Configuration:**

Epoch	Learning Rate	Batch Size	Train Loss	Test Loss	Train Accuracy	Test Accuracy	Cross Entropy Function
150	0.0001	128	0.9661	1.1958	64.87%	57.48%	AdamW(weight_decay=0.001)

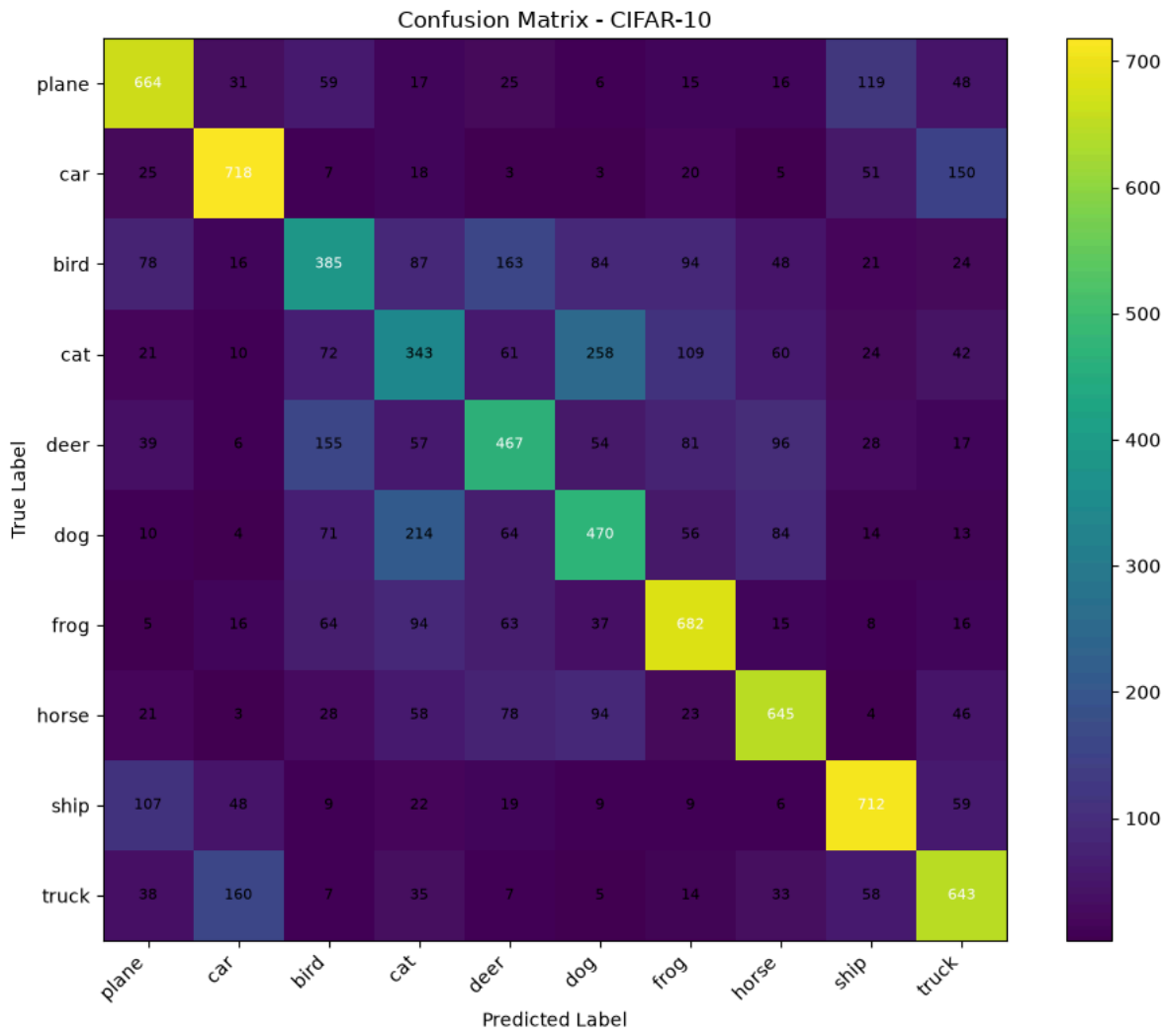


The training progress of GRU Model

The table accuracy of **10 classes** based on the **Test Dataset**

airplane	**65.5%**
automobile	**71.7%**
bird	**39.0%**
cat	**33.7%**
deer	**47.1%**
dog	**45.8%**
frog	**67.8%**
horse	**63.8%**
ship	**71.1%**
truck	**63.4%**

Confusion Matrix



The Confusion Matrix of GRU Model

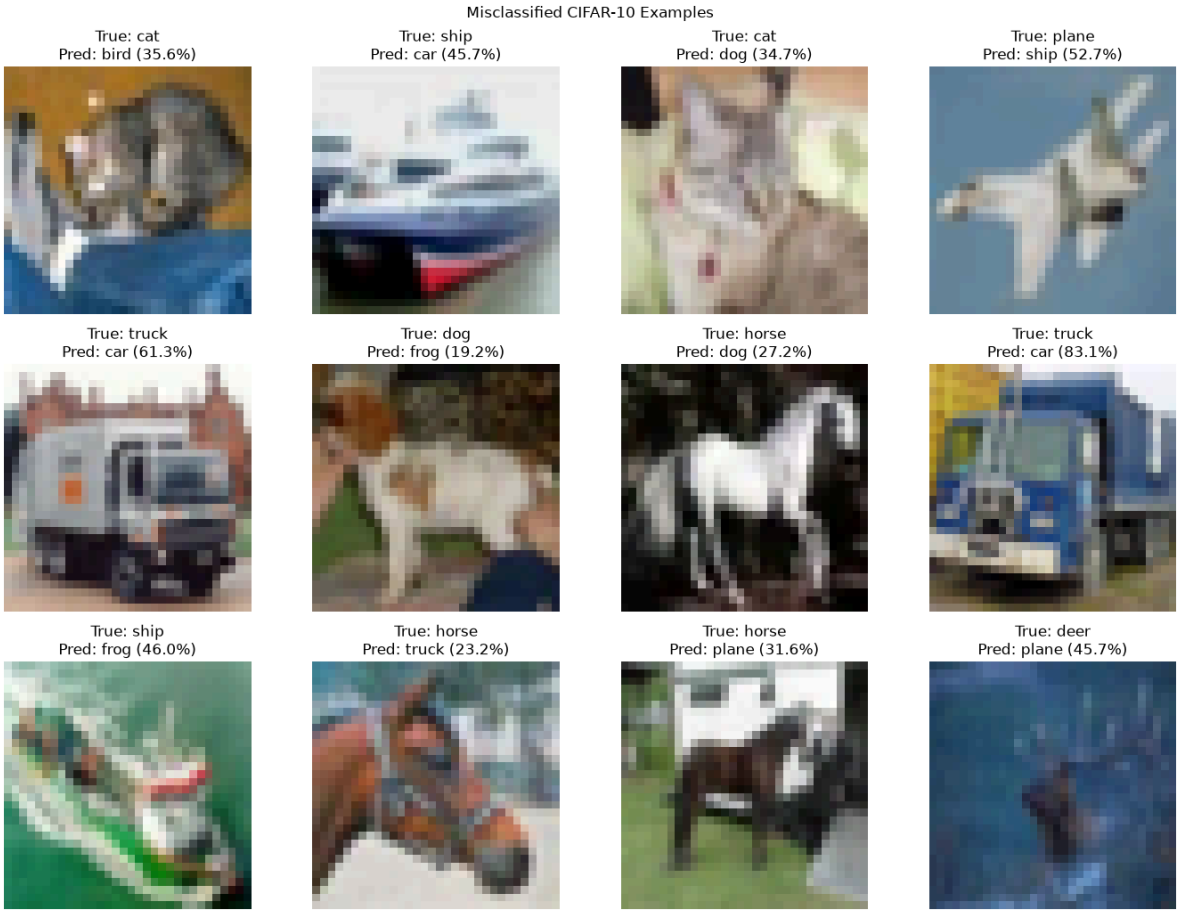
Classification Metric

classes	precision	recall	f1-score	samples
airplane	0.66	0.66	0.66	1000
automobile	0.71	0.72	0.71	1000
bird	0.45	0.39	0.41	1000
cat	0.36	0.34	0.35	1000
deer	0.49	0.47	0.48	1000
dog	0.46	0.47	0.47	1000
frog	0.62	0.68	0.65	1000
horse	0.64	0.65	0.64	1000

ship	0.69	0.71	0.70	1000
truck	0.61	0.64	0.62	1000

	precision	**recall**	**f1-score**	**samples**
accuracy			0.57	10000
macro avg	0.57	0.57	0.57	10000
weighted avg	0.57	0.57	0.57	10000

Display Misclassified Images based on Classes:



2.3.3 The Comparison And Evaluation Between LSTM And GRU

	LSTM	**GR**
Learning Curve	**LSTM overfits on training data:** LSTM achieves	**GRU shows better regularization:** GRU's

	higher training accuracy (**66.88%**), but suffers a performance drop of **11.84%** on test data due to parameter over-capacity for raw pixel/sequential image data.	lower cell complexity (combining hidden state and cell state, using reset/update gates) acts as an implicit regularizer. It limits overfitting, keeping the test loss to **1.1958** compared to LSTM's **1.3001**.
Classification Metric	Across almost all classes (especially automobile, cat, deer, dog, frog, truck), LSTM exhibits substantially higher recall scores (e.g., Cat Recall: **0.52 vs 0.34**; Truck Recall: **0.74 vs 0.64**). LSTM predicts positive instances more liberally.	**GRU favors precision:** GRU achieves higher precision on major classes (automobile, frog, horse, truck), resulting in cleaner, fewer false-positive predictions.
Overall	The **GRU model outperforms the LSTM model** in overall generalization, achieving higher test accuracy (**57.48% vs 55.04%**) and lower test loss (**1.1958 vs 1.3001**), while exhibiting significantly less overfitting. However, LSTM demonstrates higher recall across most individual classes.	

2.3.4 The Comparison And Evaluation Between LSTM And MLP

	LSTM	**MLB**
Learning Curve	- **LSTM** achieves significantly lower training loss (**0.9298 vs 1.0416**) and higher training accuracy (**66.88% vs 63.00%**), demonstrating that recurrent connections allow the model to fit high-dimensional patterns better than a standard Multi-Layer Perceptron. **- LSTM** suffers from a higher generalization gap (**11.84%**), indicating overfitting as training progresses past epoch 100. - **MLP** exhibits underfitting with higher training loss, though its simpler architecture keeps the generalization gap narrower (**9.00%**).	
Classification Metric	**LSTM** significantly outperforms MLP on difficult animal classes, such as cat and bird, where spatial deformation and	**MLP** matches or slightly edge out LSTM in deer, truck, and frog, where static pixel intensities correlate directly with global feature

	varied backgrounds confuse flat dense networks	templates.

2.3.5 The Comparison And Evaluation Between LSTM And CNN

	LSTM	CNN
Learning Curve	**CNN** achieves an outstanding **1.70%** generalization gap compared to LSTM's **11.84%**. CNN maintains test performance almost identical to its training accuracy, proving that 2D spatial feature extraction avoids memorization of raw input pixel sequences. **CNN** achieves a test accuracy of **64.62%** (+9.58% higher than LSTM) and drops test loss below 1.0 (**0.9983 vs 1.3001**), demonstrating higher prediction confidence and fewer high-loss misclassification penalties.	
Classification Metric	- CNN outperforms LSTM in Precision across **all 10 classes**, achieving a macro precision of **0.65 vs 0.55** (+0.10). The largest precision gains occur in rigid object categories: truck (**+0.17**), automobile (**+0.14**), frog (**+0.12**), airplane (**+0.09**), and horse (**+0.09**). - CNN's translation invariance and local receptive fields allow it to significantly suppress false-positive predictions without sacrificing recall (CNN matches **LSTM** recall across 9 of 10 categories). **- CNN is** decisively superior to **LSTM** for image data classification tasks, producing higher accuracy, cleaner precision bounds, and robust regularization against overfitting.	

2.3.5 The Comparison And Evaluation Between GRU And MLP

	GRU	MLP
Learning Curve	**-** CNN outperforms GRU by **+7.14%** in test accuracy (**64.62% vs 57.48%**). **-** CNN demonstrates a tight generalization gap of only **1.70%** compared to GRU's **7.39%**. CNN's 2D local receptive fields preserve spatial structures without overfitting to sequential scan patterns. **Lower Test Error Penalty:** CNN keeps test loss below 1.0 (**0.9983 vs 1.1958**), reflecting higher model confidence and fewer severely penalizing misclassifications.	
Classification Metric	**F1-Score:** GRU outperforms or equals MLP in **9 out of 10 classes**, achieving a macro average F1-score of **0.57** vs.	

	MLP's **0.54**. **Precision:** GRU reaches a macro precision of **0.57** (peaking at **0.71** for automobile), whereas MLP achieves **0.54** (peaking at **0.64** for ship). **Recall:** GRU maintains higher sensitivity across categories (macro recall **0.57 vs 0.54**), substantially outperforming MLP in vehicle detection (automobile recall: **0.72 vs 0.66**; truck recall: **0.64 vs 0.60**).

2.3.6 The Comparison And Evaluation Between GRU And CNN

	GRU	**CNN**
Learning Curve	**- CNN** achieves an outstanding **1.70%** generalization gap compared to LSTM's **11.84%**. CNN maintains test performance almost identical to its training accuracy, proving that 2D spatial feature extraction avoids memorization of raw input pixel sequences. **- CNN** achieves a test accuracy of **64.62%** (+9.58% higher than LSTM) and drops test loss below 1.0 (**0.9983 vs 1.3001**), demonstrating higher prediction confidence and fewer high-loss misclassification penalties.	
Classification Metric	- **CNN** exhibits its largest performance boosts over GRU on complex, deformable animal categories. It achieves a **+0.18** recall gain in cat (**0.52 vs 0.34**), **+0.11** in deer (**0.58 vs 0.47**), and **+0.09** in bird (**0.48 vs 0.39**). **-** Precision is higher for CNN across **every single class**, reaching peaks of **0.80** (automobile), **0.77** (ship), and **0.75** (truck). GRU generates more false positives due to spatial blurring during recurrent state propagation across pixel sequences.	

3. References

Asad, Muhammad. “Introduction to Softmax Classifier in PyTorch - MachineLearningMastery.com.” *Machine Learning Mastery*, https://machinelearningmastery.com/introduction-to-softmax-classifier-in-pytorch/. Accessed 15 October 2026.